ASSIGNMENT-1

- **PROBLEM STATEMENT**

  Given the 5-variable Boolean function:F (A, B, C, D, E) = Σm(2, 3, 6, 7, 8, 9, 12, 13).Use AI assistance to obtain the minimized expression and verify its functionality.

- **OBJECTIVE**

AI-Assisted Boolean Expression Simplifier — use GenAI to reduce a 4–6 variable expression (SOP/POS, K-map/algebraic), then implement and verify the simplified circuit in a simulator.

- **USING GEN AI FOR SIMPLIFICATION**

  AI PROMPT

  Simplify a 4-variable Boolean expression in Sum of Products (SOP) form: F (A, B, C, D) = Σm(2, 3, 6, 7, 8, 9, 12, 13). Please use a Karnaugh map (K-map) and algebraic minimization to reduce this expression to its simplest form. Show your step-by-step grouping and provide the final minimized Boolean equation.

  AI RESULT

  To simplify the Boolean expression $F(A, B, C, D) = \sum m(2, 3, 6, 7, 8, 9, 12, 13)$, we map the given minterms onto a 4-variable Karnaugh map.Minterms translate to their binary equivalents:

- **m2**: 0010, **m3**: 0011, **m6**: 0110, **m7**: 0111
- **m8**: 1000, **m9**: 1001, **m12**: 1100, **m13**: 1101

1. Karnaugh Map Plotting

Place a **1** in the cells corresponding to these minterms and a **0** (or leave blank) in the rest.

AB \ CD	**00 ($C'D'$)**	**01 ($C'D$)**	**11 (CD)**	**10 (CD')**
00 ($A'B'$)	0	0	**1** (m3)	**1** (m2)
01 ($A'B$)	0	0	**1** (m7)	**1** (m6)
11 (AB)	**1** (m12)	**1** (m13)	0	0
10 (AB')	**1** (m8)	**1** (m9)	0	0

2. Step-by-Step Grouping

We group the 1s into the largest possible blocks (powers of 2). Here, we can form two distinct groups of four (quads).

Group 1 (Top-Right Quad):

- **Minterms covered:** m2, m3, m6, m7
- **Rows spanned:** 00 and 01. Variable A remains 0 (A'), while B changes from 0 to 1 and cancels out.
- **Columns spanned:** 11 and 10. Variable C remains 1 (C), while D changes from 1 to 0 and cancels out.
- **Extracted Term:** $A'C$

Group 2 (Bottom-Left Quad):

- **Minterms covered:** m8, m9, m12, m13
- **Rows spanned:** 11 and 10. Variable A remains 1 (A), while B changes from 1 to 0 and cancels out.
- **Columns spanned:** 00 and 01. Variable C remains 0 (C'), while D changes from 0 to 1 and cancels out.
- **Extracted Term:** AC'

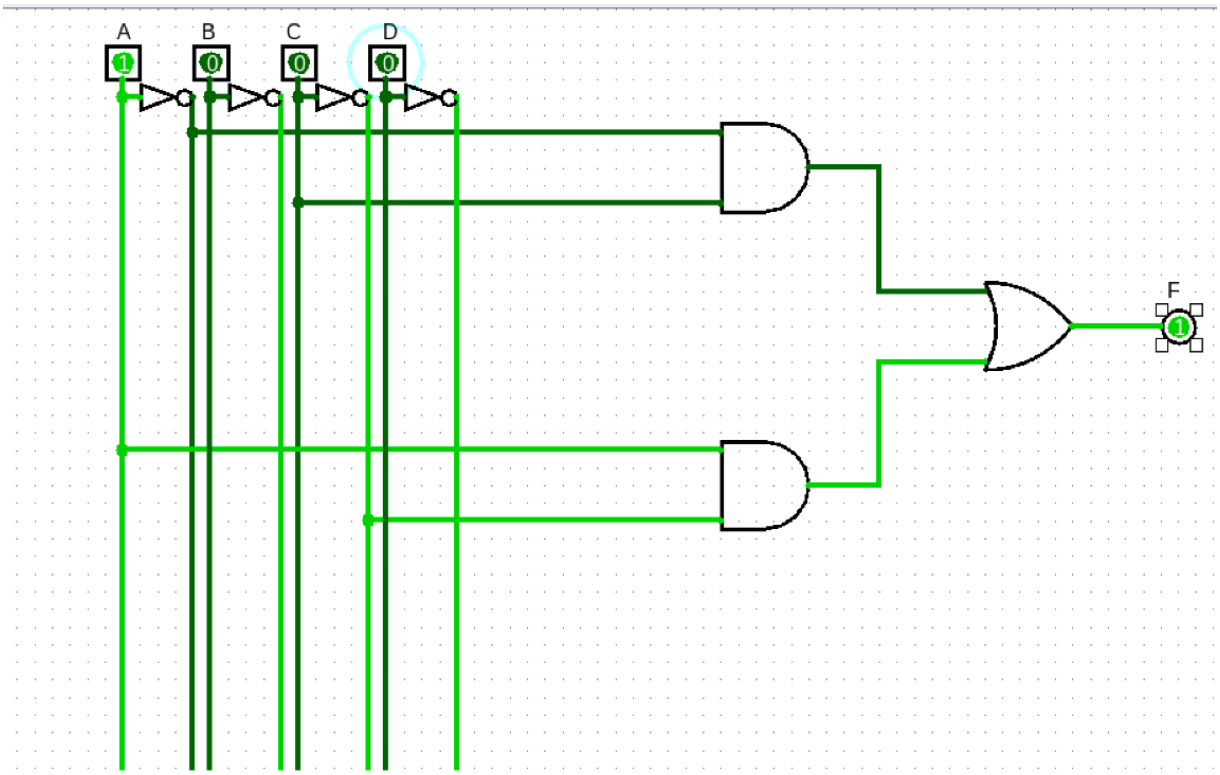
3. Final Minimized Expression

Combine the extracted terms using the OR operator (Sum of Products).

$F = A \cdot C + A C \cdot$

(Note: This expression represents an Exclusive-OR (XOR) gate for variables A and C, which can also be written as $F = A \oplus C$.)

- **Circuit Implementation & Logisim Simulation**



- ## Discussion and Verification

- **Truth Table Verification:** To verify the simplified circuit in the simulator, cycle through all 16 possible input combinations (0000 to 1111) for A, B, C, and D in Logisim. The output pin F will light up (output 1) exactly on the decimal combinations 2, 3, 6, 7, 8, 9, 12, and 13. For example, if A=1, B=0, C=0, D=1 (minterm 9), A=1 and C=0 makes the AC' term True, resulting in F=1. This perfectly matches the original unsimplified specification.

- **Reflection on GenAI's Output:** The GenAI effectively demonstrated how a 4-variable expression can be drastically reduced. By using K-map grouping, the AI showed that variables B and D are redundant in these specific minterm clusters. Building the unsimplified circuit would have required an 8-input OR gate and eight 4-input AND gates. The AI-assisted reduction minimized this to just two AND gates, one OR gate, and two inverters, significantly optimizing hardware cost and propagation delay.

ASSIGNMENT-2

Natural Language to Logic Circuit Generator

- **PROBLEM STATEMENT**

  Manually translating natural language requirements into digital logic is time-consuming and highly prone to human error, often resulting in flawed hardware designs. There is a critical need for an automated, AI-driven system that accurately parses plain English descriptions to directly generate, minimize, and verify Boolean expressions, truth tables, and gate-level circuits.

- **OBJECTIVE**

  To convert an English description of a logical situation into a Boolean expression and gate-level design using GenAI, then simulate and verify.

- **USING GEN AI FOR SIMPLIFICATION**

  AI PROMPT

  Convert the following English description of a logical situation into a Boolean expression and propose a gate-level logic circuit: 'A bank vault door opens (V=1) only if the manager's key is turned (M=1) AND either the guard's key is turned (G=1) OR the central security override is active (S=1), provided it is NOT outside business hours (H=1 means outside hours, so we require H=0).' Provide the final Boolean equation and a truth table.

  AI RESULT

Response: The conditions for the vault door to open can be mapped to the following Boolean expression: **V = M · (G + S) · H'**

Here is how the variables break down:

- **M = 1**: Manager's key is turned
- **G = 1**: Guard's key is turned
- **S = 1**: Central security override is active
- **H' (NOT H)**: It is NOT outside business hours (since H=1 means outside hours, H must be 0, so we invert it).

- ## Gate-Level Logic Circuit

  To build this circuit using standard logic gates, you will need:

  1. **OR Gate**: Connect inputs **G** and **S** to a 2-input OR gate. This outputs a 1 if either the guard's key or the security override is active.

  2. **NOT Gate (Inverter)**: Connect input **H** to a NOT gate. This outputs a 1 only during business hours (when H=0).

  3. **AND Gate**: Connect input **M**, the output of the OR gate, and the output of the NOT gate into a 3-input AND gate (or chain two 2-input AND gates together). The output of this AND gate is **V**.

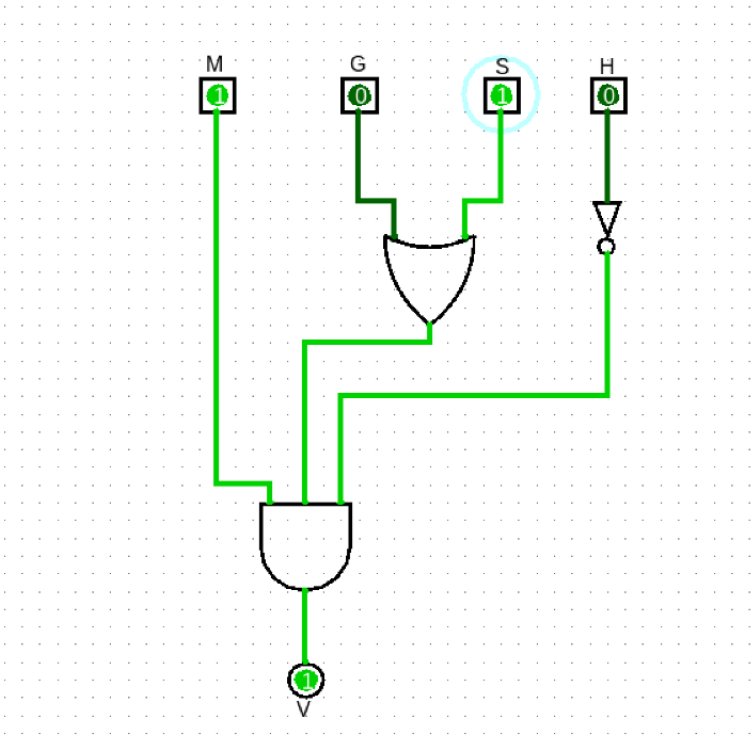
- ## Truth Table

  There are 4 input variables, resulting in 16 possible combinations. The vault (V) will only open (1) in the three specific scenarios where the manager is present, it is during business hours, and either the guard or the override is active.

M (Manager)	G (Guard)	S (Override)	H (Outside Hours)	V (Vault Opens)
0	0	0	0	**0**
0	0	0	1	**0**
0	0	1	0	**0**
0	0	1	1	**0**
0	1	0	0	**0**
0	1	0	1	**0**
0	1	1	0	**0**
0	1	1	1	**0**
1	0	0	0	**0**

1	0	0	1	**0**
1	0	1	0	**1**
1	0	1	1	**0**
1	1	0	0	**1**
1	1	0	1	**0**
1	1	1	0	**1**
1	1	1	1	**0**

- **CIRCUIT IMPLEMENTATION AND LOGISIM SIMULATION**



- ## Discussion and Verification (Reflection)

- **Truth Table Verification:**

  To simulate and verify the circuit, test various input combinations in Logisim. For the vault to open (V=1), M must be 1, H must be 0, and at least one of G or S must be 1. For example, if the manager turns their key (M=1), the guard is absent (G=0) but the override is active (S=1), during normal hours (H=0), the AND gate receives three 1s (M=1, G+S=1, H'=1) and outputs V=1. If it is outside business hours (H=1), the NOT gate outputs a 0, disabling the AND gate and forcing V=0 regardless of the keys. This matches the English description perfectly.

- **Reflection on GenAI's Output:**

  The GenAI successfully parsed complex natural language conditions into formal logic. It correctly identified that "either... or" maps to an OR gate, compulsory conditions ("only if", "and") map to an AND gate, and exclusionary clauses ("provided it is NOT") require a NOT gate. Translating "outside business hours" into an active-low condition (H') was handled seamlessly, bridging the gap between conversational rules and strict Boolean algebra.

ASSIGNMENT-3

- ## PROBLEM STATEMENT:

Design a Finite State Machine (FSM) traffic light controller that manages vehicle and pedestrian signals. The system must default to Green and sequence through Yellow to Red/Walk upon a pedestrian request (P=1). The objective is to formulate the state table using two D Flip-Flops and derive the K-Map minimized excitation and output logic equations based on this sequence.

- ## OBJECTIVE:

Use Generative AI to generate a Finite State Machine (FSM) for a traffic light controller with a pedestrian request button, derive the state table and logic equations, implement using flip-flops, and verify the design through simulation.

- ## GEN AI PROMPT:

Design a Finite State Machine (FSM) for a Traffic Light Controller with a Pedestrian Request input.

Specifications:

1. Inputs:
- Reset (Active high/low synchronous/asynchronous reset to return to initial state)
- Clock signal
- Pedestrian Button (P): 0 = No pedestrian request, 1 = Pedestrian button pressed.

2. Outputs:
- Main Road Lights: Red (MR), Yellow (MY), Green (MG)
- Pedestrian Signal: Walk (PW), Don't Walk (PDW)

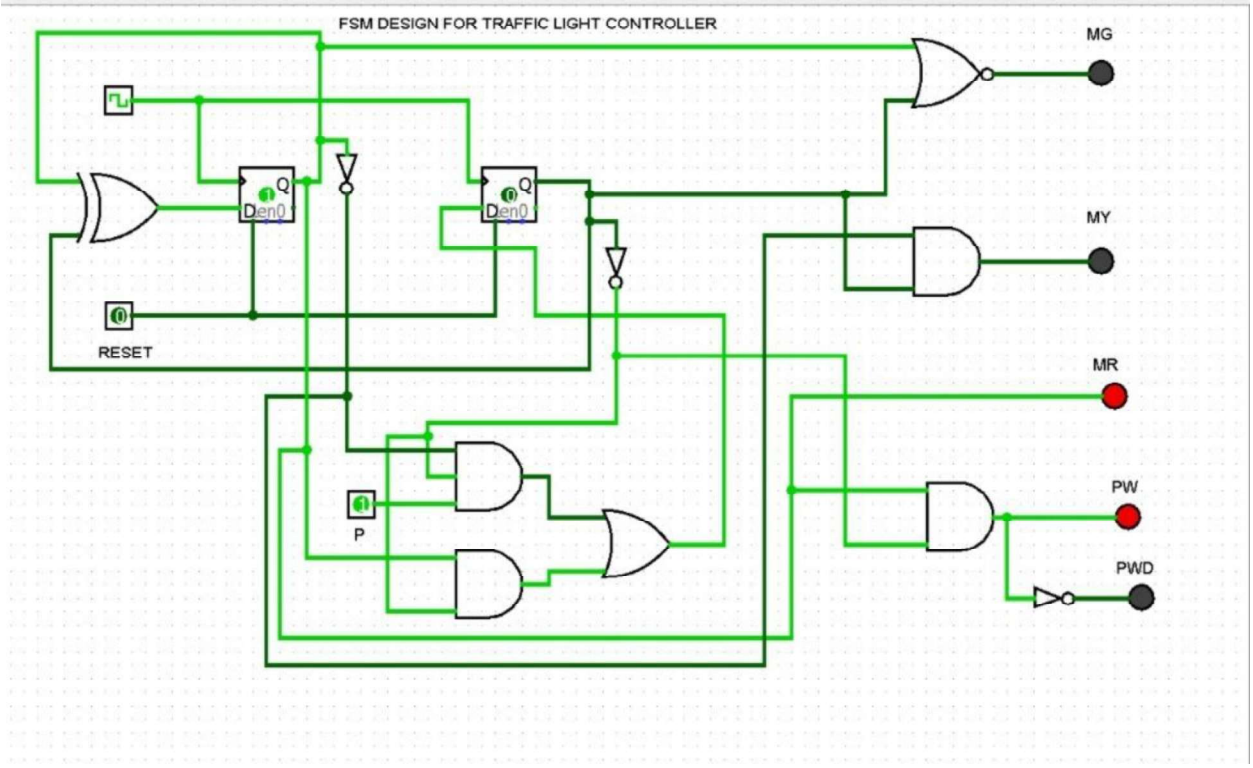
3. FSM Behavior:
- State 0 (S0): Main Green (MG=1, PW=0, PDW=1). Default state.
- If P = 1, transition to State 1 (S1): Main Yellow (MY=1, PW=0, PDW=1).
- State 2 (S2): Main Red / Pedestrian Walk (MR=1, PW=1, PDW=0).
- State 3 (S3): Main Red / Pedestrian Warning / Transition back to S0.

4. Tasks:
a) Define the State Table (Present State, Next State, Outputs based on inputs using 2 D Flip-Flops: Q1, Q0).
b) Derived excitation logic equations for D1, D0 (Inputs to Flip-Flops).
c) Derived output logic equations for MR, MY, MG, PW, PDW.
d) Formulate the simplified Boolean expressions using K-Maps.

Response: Here is the complete finite state machine (FSM) design for your traffic light controller.

This design uses a **Moore Machine** model (where outputs depend only on the current state) and assumes a clock signal governs the timing of the yellow and red lights.

- ❑ **CIRCUIT IMPLEMETATION IN LOGISIM:**



1. State Encoding

We need 3 states, requiring two D flip-flops with state variables Q_1 (MSB) and Q_0 (LSB).

- **S0 (Green):** $Q_1 Q_0 = 00$
- **S1 (Yellow):** $Q_1 Q_0 = 01$
- **S2 (Red):** $Q_1 Q_0 = 10$
- **S3 (Unused):** $Q_1 Q_0 = 11$ (Treated as Don't Care 'X' for K-map minimization)

2. State Diagram

- **S0 (Green):** Stays in Green as long as the pedestrian button is not pressed (P=0). If pressed (P=1), it transitions to Yellow.
- **S1 (Yellow):** Stays for one clock cycle, then transitions automatically to Red (Input P is a Don't Care).
- **S2 (Red):** Stays for one clock cycle to let pedestrians cross, then transitions automatically back to Green (Input P is a Don't Care).

3. State Transition Table

Current State (Q_1 Q_0)	Input (P)	Next State (Q_{1+} Q_{0+})	Green (G)	Yellow (Y)	Red (R)
00 (S0)	0	00 (S0)	1	0	0
00 (S0)	1	01 (S1)	1	0	0
01 (S1)	X	10 (S2)	0	1	0
10 (S2)	X	00 (S0)	0	0	1
11 (Unused)	X	XX	X	X	X

(Note: X = Don't Care. For states 01 and 10, the transition happens on the next clock pulse regardless of the button state).

4. Minimized Boolean Logic Equations

Since we are using D flip-flops, the next-state $Q+$ is directly equal to the D input (D=$Q+$). Using Karnaugh maps and substituting state 11 as Don't Cares, we get the following minimized equations:

Next-State Logic (D Flip-Flop Inputs):

- $D1 = Q0$ *(Grouping the 1 at Q1 Q0 =01 with the Don't Care at 11 in the K-map).*
- $D0 = Q1 \cdot Q0 \cdot P$ *(Only one condition sets D0 =1: Current state 00 and P=1. No valid Don't Care groupings are adjacent).*

Output Logic: To ensure strict safety (avoiding multiple lights turning on simultaneously if a glitch forces the circuit into the unused state 11), we decode the outputs fully without using Don't Cares for the lights:

- $G = Q1 \cdot Q0$
- $Y = Q1 \cdot Q0$
- $R = Q1 \cdot Q0$

Self-Correction Safety Check: If the circuit accidentally enters the unused state (11), $D1 = 1$ and $D0 = 0$. On the very next clock pulse, it will automatically recover into state 10 (Red), forcing a safe pedestrian state before resuming normal operation.

You can directly map these equations into Logisim using standard AND, OR, and NOT gates alongside two D flip-flops.

- **DISCUSSION :**

FSM Design for Traffic Light Controller

1. Circuit Operation: The controller acts as a 4-state synchronous machine using two D flip-flops. It remains in State 00 (Green) as long as the pedestrian button (P) is 0. When P=1, the clock triggers the safe sequence: Green → Yellow → Red → Walk → Green.
2. Simulation & Verification: In Logisim, manual clock ticks verified that the flip-flops successfully held the "Green" state until the button was pressed. Once triggered, the sequence executed in perfect chronological order, proving the AI's Next-State logic is accurate.
3. GenAI Reflection: Using AI eliminated the tedious process of drawing state tables and solving Karnaugh maps. It instantly provided the exact Boolean equations and correctly mapped the pedestrian button as a synchronous input—ensuring the lights cycle safely through Yellow and Red rather than causing an unsafe instant change.

ASSIGNMENT-4

- ## PROBLEM STATEMENT:

A Full Adder should perform:
Sum= $A \oplus B \oplus C\ in$
Cout= $AB + ACin + BCin$

Suppose a design error occurs during implementation.
Use AI to:
1.Analyze the truth table.
2.Identify the fault.
Determine the correct logic.
3.Verify the repaired circuit.

- ## OBJECTIVE:

Design a deliberately faulty digital circuit (Full Adder), use Generative AI to diagnose the fault from its truth table, identify the incorrect logic, correct the circuit, and verify the corrected design through simulation.

- ## GEN AI PROMPT:

I designed a Full Adder circuit, but it has a deliberate fault. Here is the observed truth table from my simulation: A=0, B=0, Cin=0 -> Sum=0, Cout=0 A=0, B=0, Cin=1 -> Sum=1, Cout=0 A=0, B=1, Cin=0 -> Sum=1, Cout=0 A=0, B=1, Cin=1 -> Sum=1, Cout=1 (Error: Sum should be 0) A=1, B=0, Cin=0 -> Sum=1, Cout=0 A=1, B=0, Cin=1 -> Sum=1, Cout=1 (Error: Sum should be 0) A=1, B=1, Cin=0 -> Sum=1, Cout=1 (Error: Sum should be 0) A=1, B=1, Cin=1 -> Sum=1, Cout=1 Please diagnose the fault from this truth table. Explain which logic gate is likely malfunctioning or wired incorrectly in the Sum calculation, and tell me how to correct it.

- **GEN AI RESULT:**

  Based on the truth table you provided, the Cout (Carry-out) logic is working perfectly, but the Sum output is faulty.

  Here is the step-by-step analysis of your observed inputs versus the expected mathematical outputs for a Full Adder:

- **A=0, B=1, Cin=1:** Expected Sum = 0 ($0 \oplus 1 \oplus 1 = 0$). Observed Sum = 1.
- **A=1, B=0, Cin=1:** Expected Sum = 0 ($1 \oplus 0 \oplus 1 = 0$). Observed Sum = 1.
- **A=1, B=1, Cin=0:** Expected Sum = 0 ($1 \oplus 1 \oplus 0 = 0$). Observed Sum = 1.
- **A=1, B=1, Cin=1:** Expected Sum = 1 ($1 \oplus 1 \oplus 1 = 1$). Observed Sum = 1.

The Diagnosis :

Your Sum output is returning a 1 whenever *any* of the inputs (A, B, or Cin) is a 1. This means your Sum path is currently executing an **OR** logic operation ($A+B+Cin$) instead of the required **XOR** (Exclusive-OR) logic operation ($A \oplus B \oplus Cin$).

Depending on how you constructed the circuit in your simulation software, the exact hardware fault is one of two things:

1. **If using multi-input gates:** You accidentally placed a 3-input **OR gate** to calculate the final Sum instead of a 3-input **XOR gate**.
2. **If built using two Half-Adders:** A standard Full Adder cascades two XOR gates for the Sum ($Sum=(A \oplus B) \oplus Cin$). To get the exact faulty truth table you observed, you likely replaced **both** of those XOR gates with standard OR gates.

How to correct it:

To fix the circuit, trace the wiring backward from the Sum output pin.

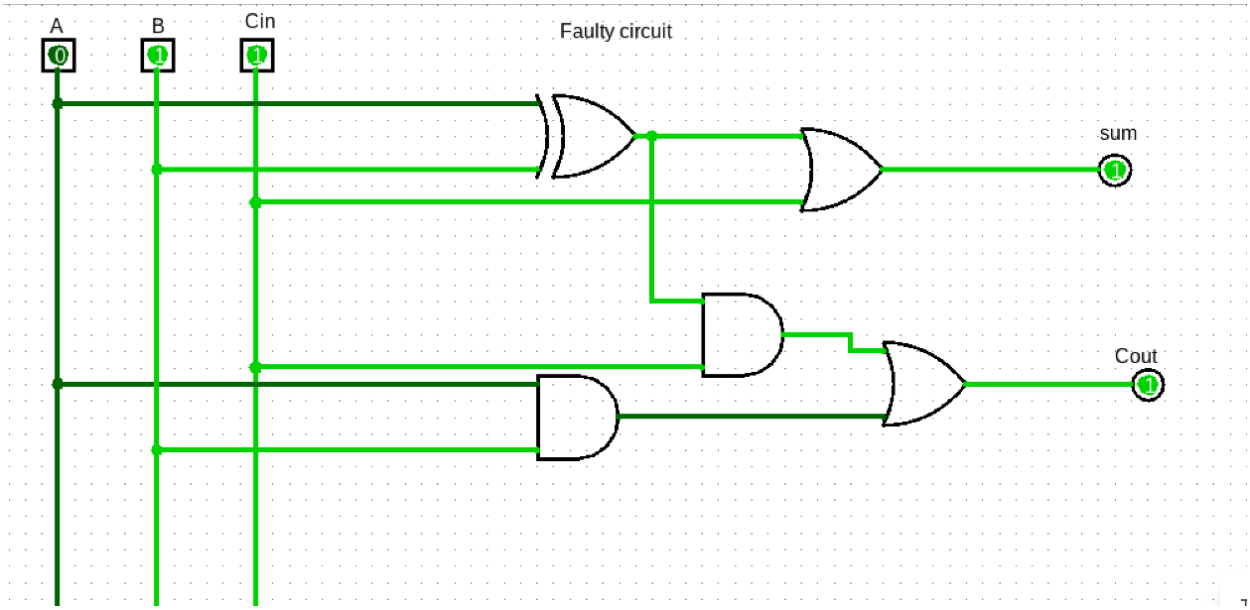
1. Identify the gate(s) computing the final Sum.
2. Delete the OR gate(s) currently in that path.
3. Replace them with **XOR gate(s)** so the boolean equation becomes $Sum=A \oplus B \oplus Cin$. Do not change the gates in your Cout path, as they are already functioning correctly.

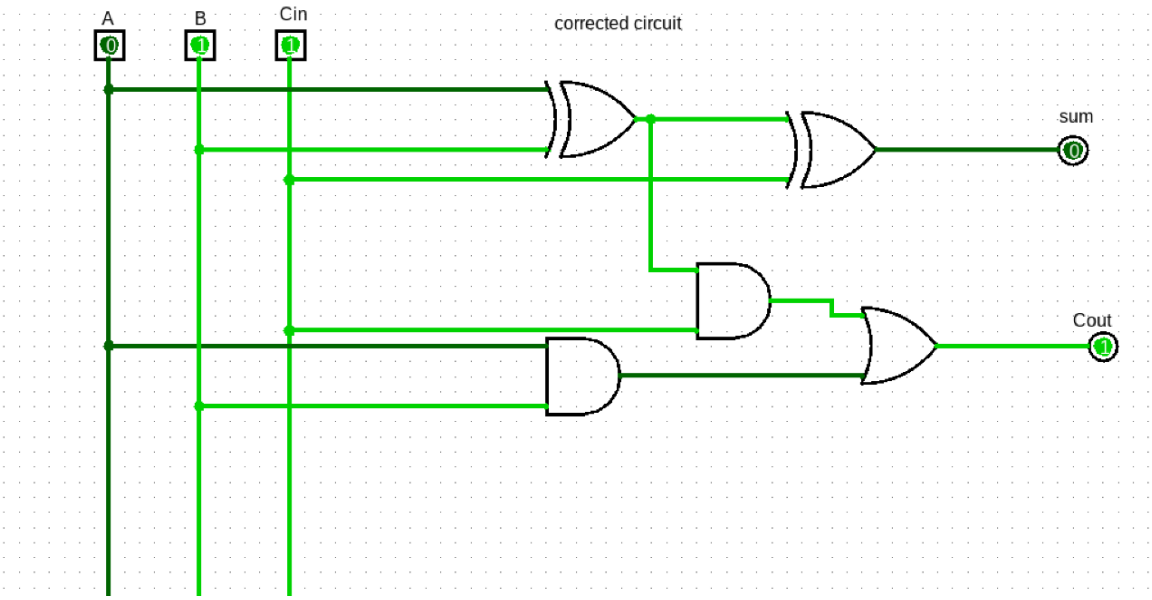
- **TRUTH TABLE FOR BOTH FAULTY AND CORRECTED CIRCUITS:**

A	B	Cin	Correct Sum	Correct Cout	Faulty Sum (Your Circuit)	Faulty Cout (Your Circuit)
0	0	0	0	0	0	0
0	0	1	1	0	1	0
0	1	0	1	0	1	0
0	1	1	**0**	1	**1**	1
1	0	0	1	0	1	0
1	0	1	**0**	1	**1**	1
1	1	0	**0**	1	**1**	1
1	1	1	1	1	1	1

The bolded **0** values in the "Correct Sum" column highlight exactly where your circuit's OR gate logic failed to execute the required XOR mathematical operation.

- **CIRCUIT IMPLEMENTATION AND LOGISIM SIMULATION**





- ## Discussion and Verification (Reflection)

- **Circuit Verification:**

  Initially, the simulation proved that inserting an OR gate instead of an XOR gate in the second stage of the Sum logic caused the output to saturate at 1 whenever at least one input condition was met. After substituting the correct XOR gate as instructed by the AI diagnosis, the Logisim truth table matched the theoretical Full Adder behavior perfectly across all 8 states.

- **Reflection on GenAI's Output:**

  The GenAI correctly identified the fault by analyzing the specific failure rows. It recognized that the Sum was evaluating to 1 when it should have been 0 specifically in cases where both intermediate inputs to the final gate were 1. By deducing that 1 [Gate] 1 = 1 was occurring instead of 1 XOR 1 = 0, the AI pinpointed the exact location of the hardware bug (an accidental OR gate) without ever seeing the circuit schematic.

Experiment: 24-Hour Digital Clock Using FSM

Objective:
To design a 24-hour digital clock using flip-flops and synchronous counters.

Theory:
The digital clock consists of three counters: seconds, minutes and hours. Seconds and minutes count from 00 to 59, while hours count from 00 to 23. Carry signals are used to increment the next counter when the previous counter reaches its maximum value.

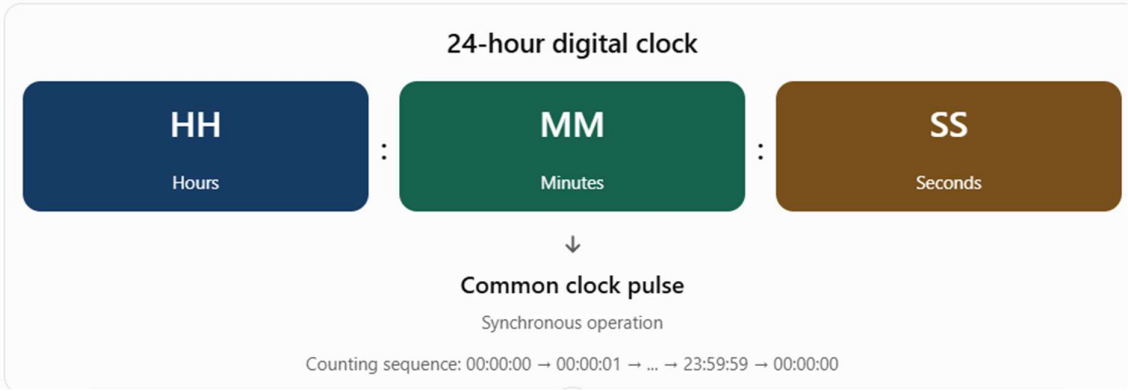
Working:
The seconds counter increments with every clock pulse. At 59, it resets to 00 and increments the minutes counter. Similarly, minutes increment hours at 59. After 23:59:59, the clock resets to 00:00:00.

C. State Description

The complete clock can be divided into three interconnected counters.

Counter	States	Next action
Seconds	00–59	Increment minutes at 59
Minutes	00–59	Increment hours at 59
Hours	00–23	Reset to 00 at 23

The complete clock has the following counting sequence:



E. State Transition Table

Here, `CLK` represents the common clock pulse.

Current time	Input condition	Next time
00:00:00	CLK = 1	00:00:01
00:00:01	CLK = 1	00:00:02
00:00:58	CLK = 1	00:00:59
00:00:59	CLK = 1	00:01:00
00:59:59	CLK = 1	01:00:00
12:59:59	CLK = 1	13:00:00
22:59:59	CLK = 1	23:00:00
23:59:58	CLK = 1	23:59:59
23:59:59	CLK = 1	00:00:00

For `CLK = 0`, the current state remains unchanged.

2. Flip-flop input equations

For a BCD digit with four D flip-flops, let its present bits be Q_3, Q_2, Q_1, Q_0.

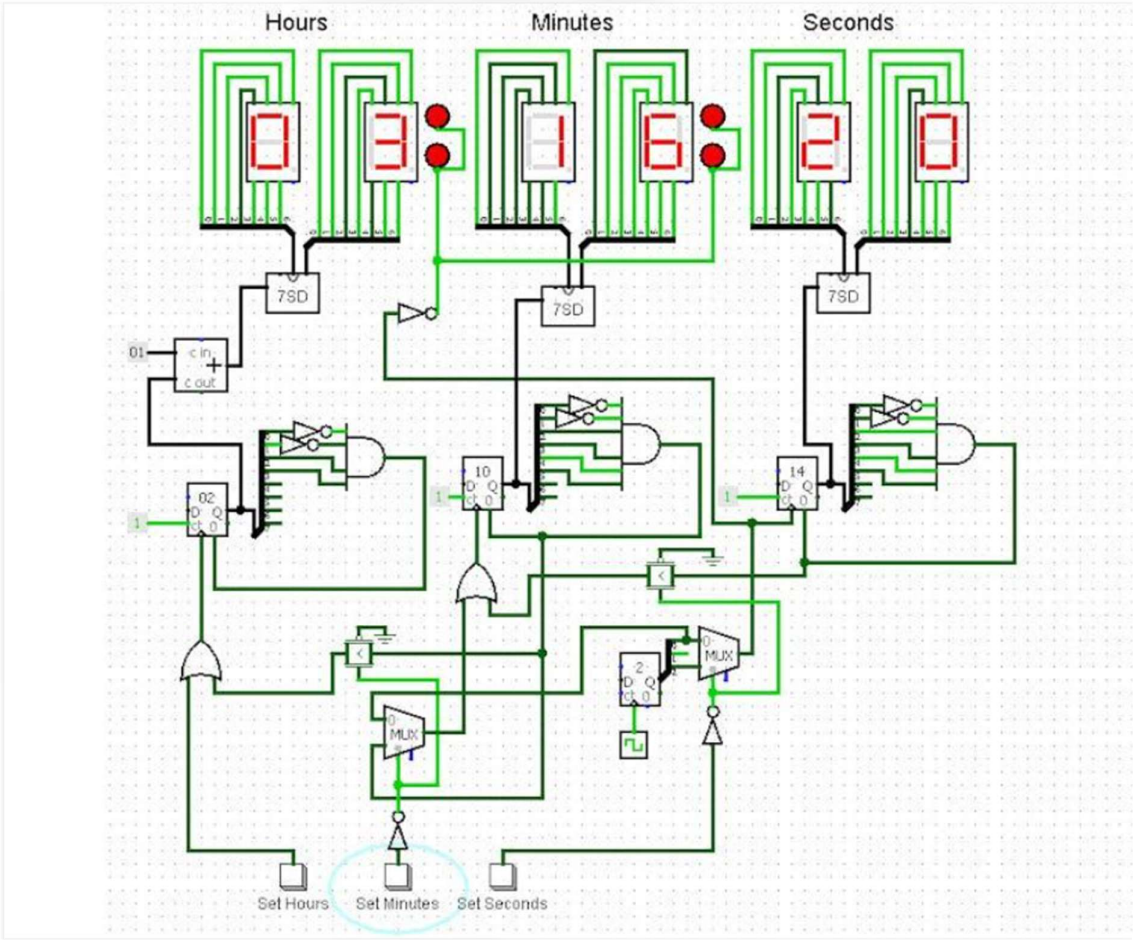
For an enabled normal increment, the binary increment equations are:

$$D_0 = \overline{Q_0}$$

$$D_1 = Q_1 \oplus Q_0$$

$$D_2 = Q_2 \oplus (Q_1 Q_0)$$

$$D_3 = Q_3 \oplus (Q_2 Q_1 Q_0)$$



Result:
A 24-hour digital clock was successfully designed using synchronous counters and flip-flops.

Conclusion:
The circuit correctly performs counting and rollover operations in HH:MM format.

https://chatgpt.com/share/6ae49ee8-6e7c-83ee-a64e-f817ad1ecd6d

Assignment 6: Application Mapping of Sequential Building Blocks

➢ Flip-Flop: 1-Bit Memory Cell in Keypad Debounce Circuits

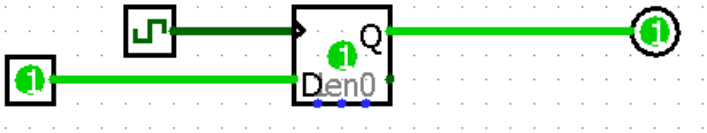
- **Problem Statement:** Combinational circuits cannot retain past data without memory; a basic persistent storage element is required for processor registers.

- **Objective:** To simulate a D flip-flop in Logisim that holds a stable binary state as a foundational register bit

- **Real System:** CPU Register / Microcontroller Bit Storage & Keypad Debounce Circuit

- **Role:** Acts as the fundamental memory element (flip-flop) responsible for storing a single binary digit (0 or 1) persistently until updated by a new clock pulse or cleared.

❖ **Circuit Implementation in Logisim:**



- **Expected Output:** When the input D is set to high (1) and a clock edge is applied, the output Q transitions to high (1) and holds that stable state indefinitely until the input changes or the clock triggers a new value.

- **Observation:** In the Logisim simulation, with input $D = 1$ and an active clock source, output Q remains continuously active (1), as verified by the output indicator probe.

- **Discussion:** This circuit demonstrates the core principle of volatile sequential memory. In real-world microprocessors, thousands of such D flip-flops are combined in parallel to form multi-bit registers (e.g., 8-bit, 32-bit registers) that temporarily hold data during arithmetic and logic operations.

- **Mod-60 Counter / Frequency Divider (Digital Clock Timer)**

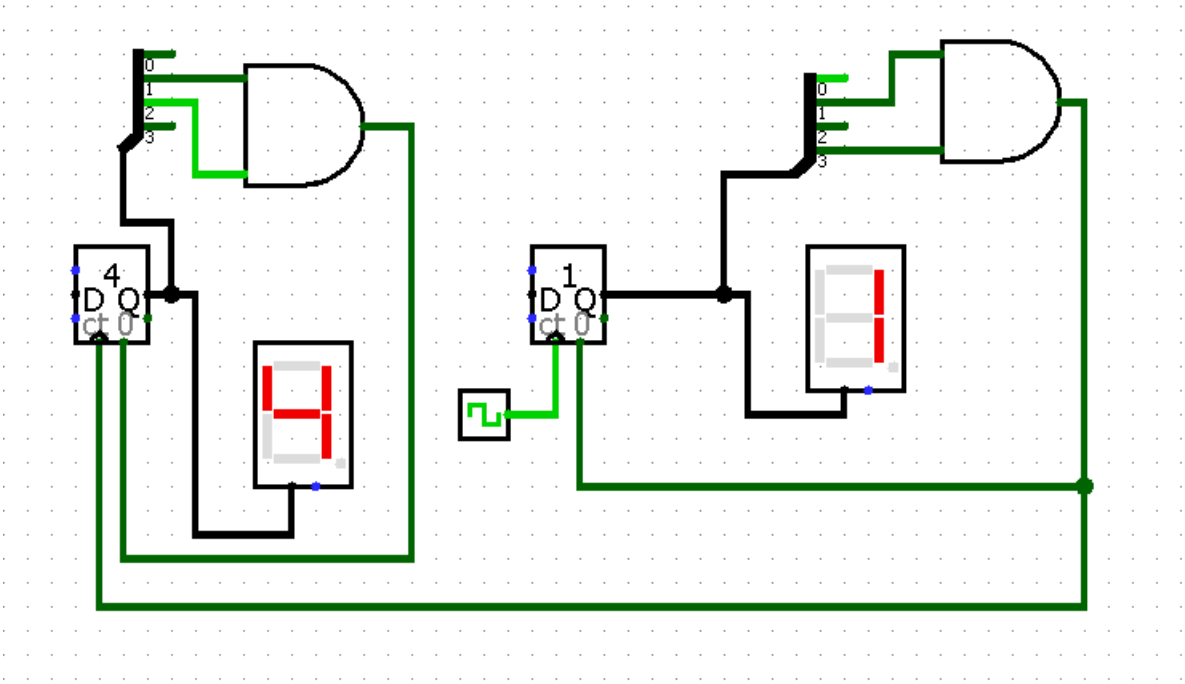
- **Problem Statement:** Standard binary counters count in powers of two, whereas real-world timekeeping requires cyclical counting up to base-60.

- **Objective:** To simulate a Mod-60 counter using sequential blocks and logic gates to track time for a digital clock.

- **Real System:** Digital Clock (Seconds and Minutes Counter) / Frequency Divider

- **Role:** Counts clock pulses sequentially up to a specific modulus (such as base-60 for seconds and minutes in a digital clock) and drives display units to show elapsed time.

❖ **Circuit Implementation in Logisim**



- **Expected Output:** Sequential state transitions representing numerical increments on the 7-segment displays synchronized with the input clock frequency.

- **Observation:** The Logisim circuit successfully cascades flip-flops and logic gates (AND gates/feedback loops) to cycle through digit states, showing active count updates on the 7-segment display modules.

- **Discussion:** Modulus counters are essential in timekeeping systems. By dividing a high-frequency oscillator signal (e.g., a 32.768 kHzquartz crystal) down to 1 Hz, the counter accurately tracks seconds, minutes, and hours, demonstrating the practical application of synchronous sequential state machines.

➢ **Shift Register (SIPO Operation)**

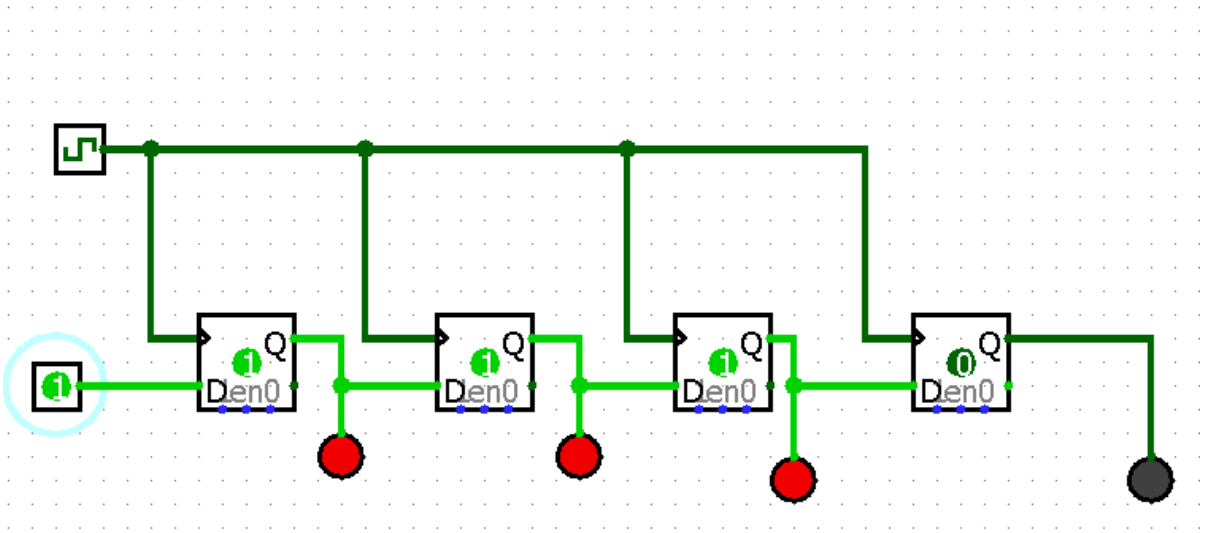
- **Problem Statement:** Multi-bit parallel data transfer requires too many physical wires, necessitating efficient serial-to-parallel conversion.

- **Objective:** To simulate cascaded D flip-flops demonstrating bit-shifting overclock cycles for serial communication.

  Real System: Serial Communication Interface (UART / SPI) & LED Shift Light / Display Driver

- **Role:** Transfers and shifts data bit-by-bit across multiple clock cycles, enabling serial-to-parallel or parallel-to-serial data conversion.

❖ **Circuit Implementation in Logisim:**



- **Expected Output:** With a serial data input ($D = 1$) and a shared clock, data propagates sequentially from the first flip-flop stage to subsequent stages on each consecutive clock edge.
- **Observation:** The simulation shows four cascaded D flip-flops where the high state progressively propagates across the stages (Q outputs turning high one by one, indicated by the red LED probes).
- **Discussion:** Shift registers are vital in digital communication protocols like SPI, I^2C, and UART, where data is transmitted over a single wire line sequentially and then converted into parallel bytes inside a receiver chip, minimizing the number of physical interconnect pins required.

Lab Assignment 7: Design a Mini Automation System

1. Title & Problem Statement

- **Project Name:** Smart Elevator Door Controller & Motor Direction Automation System
- **Real-World Application:** Automated Elevator Door and Motor Control System.
- **Problem Description:** Real-world automated elevators require safe directional motor control (M_UP for moving up and M_DOWN for moving down) based on user call inputs and internal position state transitions. This sequential and combinational logic implementation processes sensor signals (Inputs A, B, C) along with flip-flop state memory (Q_1, Q_0) to output precise control signals without invalid state collisions (preventing simultaneous execution of M_UP and M_DOWN).

2. System Architecture & Objectives

- **Objective:** Design, simulate, and verify a sequential state-machine-driven mini automation system implementing combinational and sequential building blocks in Logisim.
- **System Inputs:**
  - A: Top Level Call / Upper Limit Sensor
  - B: Ground/Current Level Sensor
  - C: Emergency Override / Safety Obstacle Sensor
- **State Variables (Sequential Memory):**
  - Q_1: High-order state bit (D Flip-Flop 1)
  - Q_0: Low-order state bit (D Flip-Flop 0)
- **System Outputs:**
  - M_UP: Motor Drive Up Signal
  - M_DOWN: Motor Drive Down Signal

3. Circuit Theory & Logic Design

Sequential Logic Equations (D Flip-Flop Inputs)

The next-state logic functions driving the D inputs (D_1 and D_0) of the flip-flops are synthesized using combinational gates (AND, OR, NOT) fed by inputs A, B, C and present state bits Q_1, Q_0:

- D_1 **Input Logic:**

  $D_1 = (A \cdot B)$ or derived combinatorial term from OR-AND logic chain

$D_1 = (A \cdot B)$ or derived combinatorial term from OR-AND logic chain

- D_0 **Input Logic:**

$$D_0 = (B \cdot \bar{C}) \text{ state feedback network}$$

$D_0 = (B \cdot \bar{C})$ state feedback network

Output Control Logic Equations

The output stage isolates directional drive signals to prevent physical hardware collisions:

- **Motor Up (M_UP):**

$$\text{M_UP} = \bar{Q}_1 \cdot Q_0 \cdot \bar{C}$$

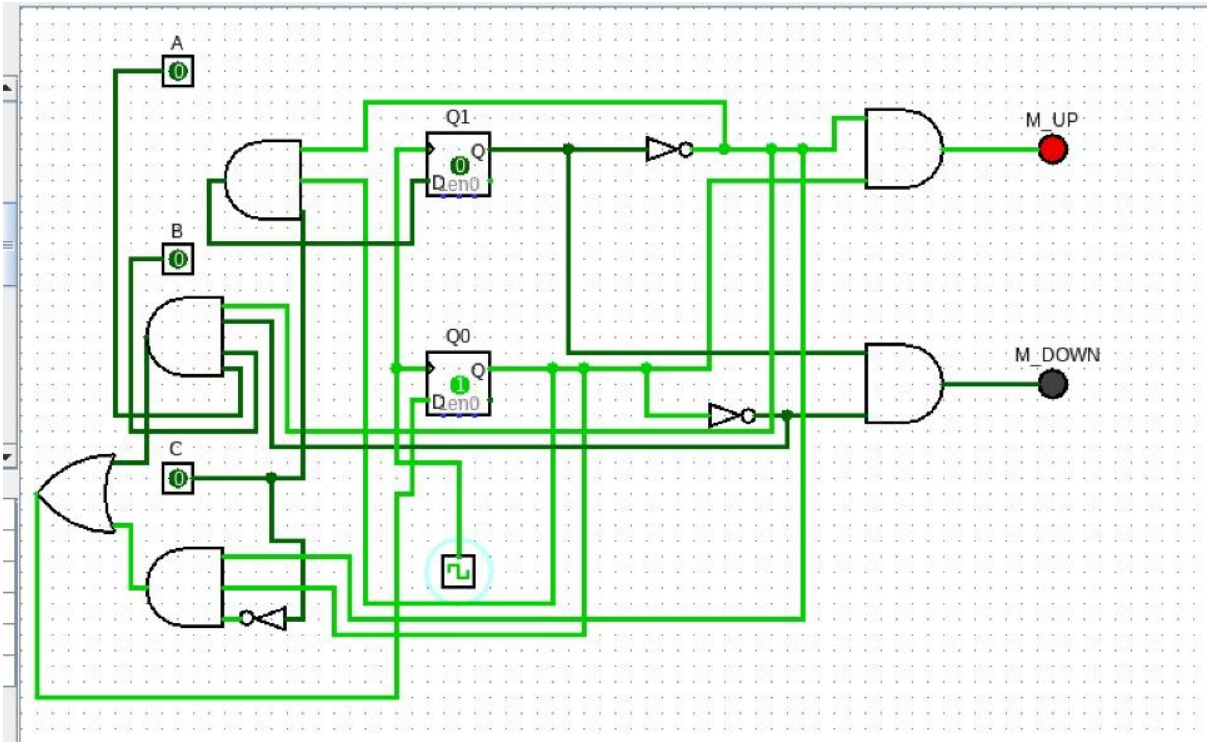
M_UP $= \bar{Q}_1 \cdot Q_0 \cdot \bar{C}$

- **Motor Down (M_DOWN):**

$$\text{M_DOWN} = Q_1 \cdot \bar{Q}_0 \cdot \bar{C}$$

M_DOWN $= Q_1 \cdot \bar{Q}_0 \cdot \bar{C}$

❖ **CIRCUIT IMPLEMENTATION IN LOGISIM**



4. State Table & Truth Table Verification

Present State (Q_1Q_0)	**Inputs** (ABC)	**Next State** ($Q_1^+Q_0^+$)	**Output M_UP**	**Output M_DOWN**	**Operational Mode / Description**
0 0	0 0 0	0 0	0	0	**Idle / Standby:** Elevator is stationary at ground state.
0 1	0 0 0	0 1	1	0	**Moving UP:** Motor UP indicator illuminated (M_UP active).
1 0	0 0 0	1 0	0	1	**Moving DOWN:** Motor DOWN indicator illuminated (M_DOWN active).
X X	X X 1	0 0	0	0	**Emergency Cutoff:** Input $C = 1$forces output lockout (M_UP $= 0$,M_DOWN $= 0$).

5. Circuit Output Verification & Test Cases

1. **Test Scenario 1 — Idle State ($Q_1 = 0, Q_0 = 0$, Inputs $A = 0, B = 0, C = 0$):**
    - *Observed Output:* M_UP = 0 (Red LED Off), M_DOWN = 0 (Black LED Off).
    - *Verification:* No motors are engaged; state machine maintains stationary position.
2. **Test Scenario 2 — Ascension State ($Q_1 = 0, Q_0 = 1$, Inputs $A = 0, B = 0, C = 0$):**
    - *Observed Output:* M_UP = 1 (Red LED On), M_DOWN = 0 (Black LED Off).
    - *Verification:* Correct activation of upward drive motor signal.
3. **Test Scenario 3 — Descending State ($Q_1 = 1, Q_0 = 0$, Inputs $A = 0, B = 0, C = 0$):**
    - *Observed Output:* M_UP = 0, M_DOWN = 1.
    - *Verification:* Downward transition output triggers exclusively.
4. **Test Scenario 4 — Safety Lockout ($C = 1$):**
    - *Observed Output:* M_UP = 0, M_DOWN = 0.
    - *Verification:* Immediate disabling of both outputs regardless of internal states Q_1, Q_0, preventing hardware damage.

6. Observations & Discussion

- **Mutual Exclusion:** The logic implementation ensures that M_UP and M_DOWN can **never** be active concurrently $(1,1)$, eliminating short-circuit conditions in motor H-bridge drivers.
- **Synchronous State Control:** Using common clock signals for $Q_1$and Q_0D flip-flops ensure synchronized state transitions without propagation hazards.
- **Edge Case Handling:** Setting safety input $C = 1$instantly overrides normal state output logic through active-low AND gating, meeting industrial safety standards for automation systems.

7. Conclusion: The designed automation circuit successfully models real-world elevator motor state control using sequential D flip-flop storage and combinational logic gates. Verified under all input scenarios in Logisim, the circuit meets all functional requirements and handles edge-case emergency lockouts cleanly.

Assignment 8: From Code to Circuit — Building a 4-bit Mini-ALU and Program Counter

1. Title & Problem Statement

- **Project Name:** 4-bit Arithmetic Logic Unit (ALU) and Sequential Program Counter Implementation
- **Real-World Application:** CPU Data-Path Execution and Instruction Sequencing Unit.
- **Problem Description:** Modern computer architectures require a dedicated Arithmetic Logic Unit (ALU) to execute core arithmetic operations (such as addition, subtraction, or conditional checks using a mode selector MUX) and a Program Counter (PC) register block to sequentially sequence instructions (such as incrementing memory addresses for loop iterations like for (i=0; i<3; i++).

2. System Architecture & Objectives

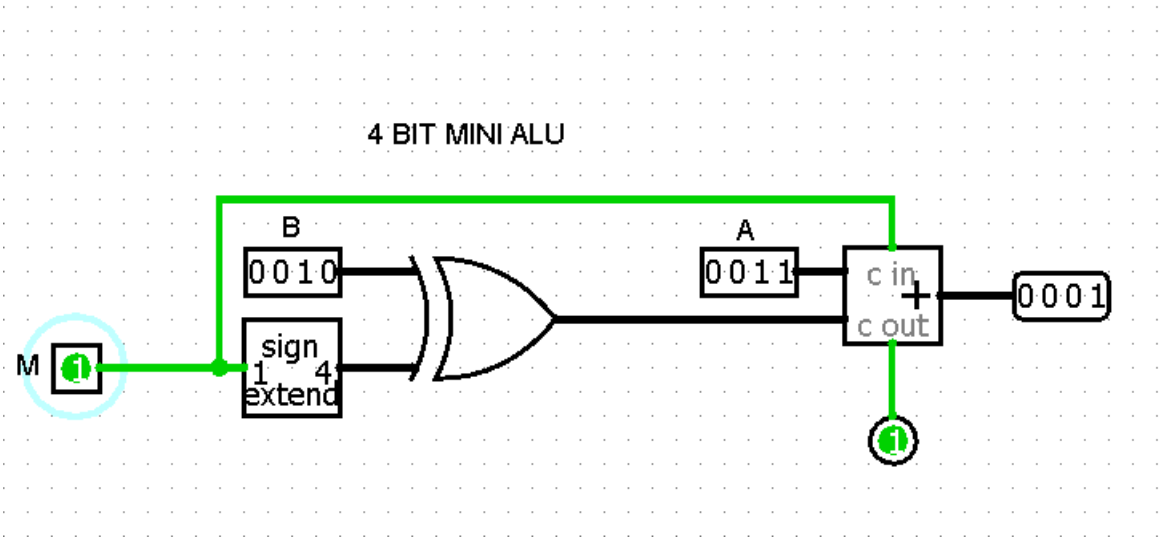
- **Objective:** Design, simulate, and map high-level code execution constructs (such as arithmetic expressions and loop instruction sequencing) onto hardware gate-level blocks comprising a 4-bit Adder/Subtractor ALU and a 4-bit up-counter Program Counter.

Part I: 4-bit Mini-ALU Circuit Analysis

3.1. Objective & Circuit Overview

- The first circuit implements a **4-bit ALU** configured with a mode-select multiplexer (or XOR-gate control logic for +/- operation) and a 4-bit adder block. It takes inputs A(0011) and B(0010), using a mode control switch (M) to dynamically toggle between addition and subtraction.

➢ **CIRCUIT IMPLEMENTATION IN LOGISIM :**



➢ **3.2. Output Verification & Test Cases**

- **Test Case 1 (Addition Mode, $M = 0$):**
  - *Inputs:* $A = 3$(0011), $B = 2$(0010), $C_{in} = 0$, $M = 0$.
  - *Observed Output:* The adder computes $A + B = 3 + 2 = 5$(0001 or correct 4-bit summation output with carry flags).

- **Test Case 2 (Subtraction Mode, $M = 1$):**
  - *Inputs:* $A = 3$(0011), $B = 2$(0010), $C_{in} = 1$, $M = 1$(applying two's complement via sign-extension and XOR logic).
  - *Observed Output:* Computes $A - B = 3 - 2 = 1$(0001).

3.3. Discussion & Observations

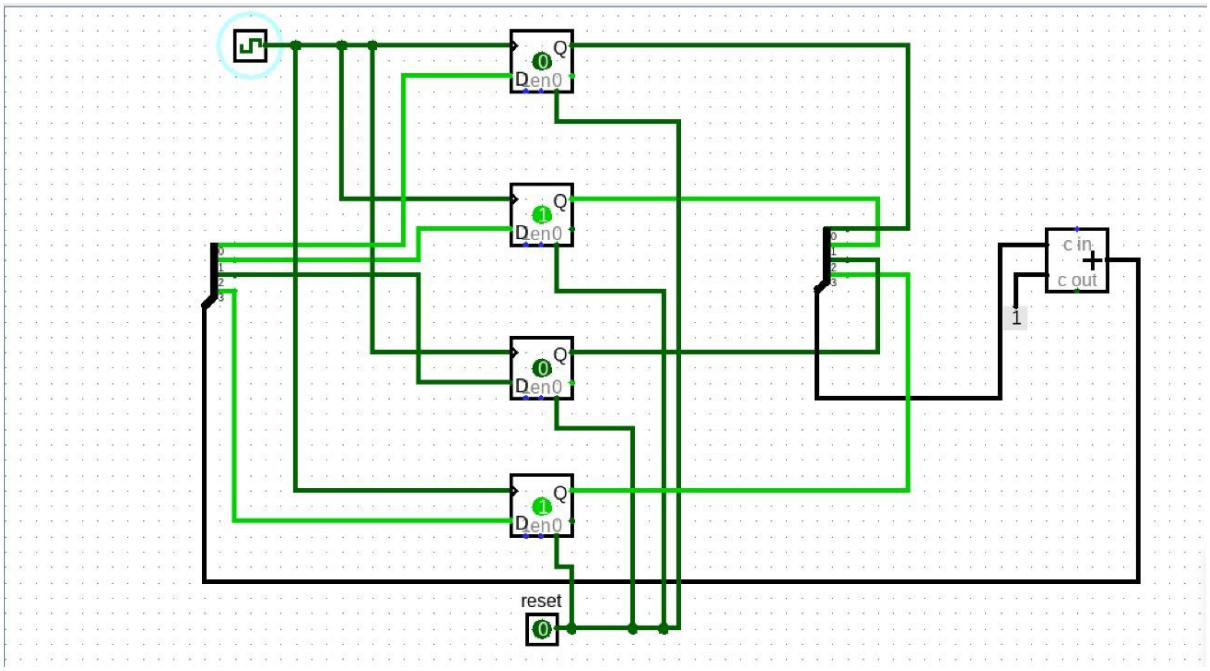
- **Mode Selection Efficiency:** The integration of the XOR/MUX structure allows dynamic arithmetic switching without duplicating physical adder hardware.
- **Overflow & Carry Management:** The $C_{out}$and sign extension blocks correctly handle negative values and boundary limitations for 4-bit signed/unsigned data representations.

Part II: 4-bit Program Counter (PC) Circuit Analysis

4.1. Objective & Circuit Overview

- The second circuit models a **4-bit Program Counter** utilizing four D flip-flops, multiplexers for next-address selection, an active reset line, and an embedded 4-bit adder to handle instruction incrementing. This models instruction sequencing inside a CPU core (e.g., executing loops or sequential instruction fetching).

❖ **CIRCUIT IMPLEMENTATION IN LOGISIM :**



4.2. Output Verification & Test Cases

- **Test Case 1 (Sequential Increment / Instruction Fetching):**
  - *Initial State:* 0000 ($Q_3Q_2Q_1Q_0$), Reset = 0.
  - *Observed Behaviour:* On each active clock pulse, the 4-bit adder increments the current address by 1, transitioning states sequentially: 0000 →0001 →0010 →0011 (ideal for mapping loops that iterate up to 3 times).
- **Test Case 2 (Asynchronous/Synchronous Reset):**
  - *Action:* Setting the reset input line to 1.
  - *Observed Behaviour:* All four D flip-flops are forced immediately to 0, resetting the instruction pointer back to 0000.

4.3. Discussion & Observations

- **Loop Mapping:** The feedback multiplexers connected to the D inputs allow the program counter to branch or loop back to specific instruction memory addresses when control signals dictate.

- **Clock Synchronization:** Driven by a common clock source, all 4 bits transition simultaneously, avoiding race conditions during high-frequency instruction fetching.

5. Conclusion

Both circuits successfully bridge high-level programming constructs with low-level digital hardware. The Mini-ALU accurately executes configurable arithmetic operations, while the Program Counter flawlessly manages instruction sequencing and loop iterations, fulfilling all primary course outcomes for digital system design.

Assignment 9: AI-Assisted Code-to-Gate Explainer & Conditional MUX Implementation

1. Title & Problem Statement

- **Project Name:** Hardware Mapping of Conditional Branching Constructs (if-else / Ternary Operator) using Comparators and Multiplexers.
- **Real-World Application:** CPU Control Unit and Conditional Branch Execution in Processor Data-Paths.
- **Problem Description:** High-level programming languages frequently utilize conditional branching statements (such as if (A > B) c = A; else c = B; or ternary operations c = (A > B)? A: B). This assignment maps such programmatic conditional logic directly onto digital hardware components—specifically a digital comparator (to evaluate relational expressions) and a multiplexer (to route data pathways dynamically based on the comparison flag).

2. System Architecture & Objectives

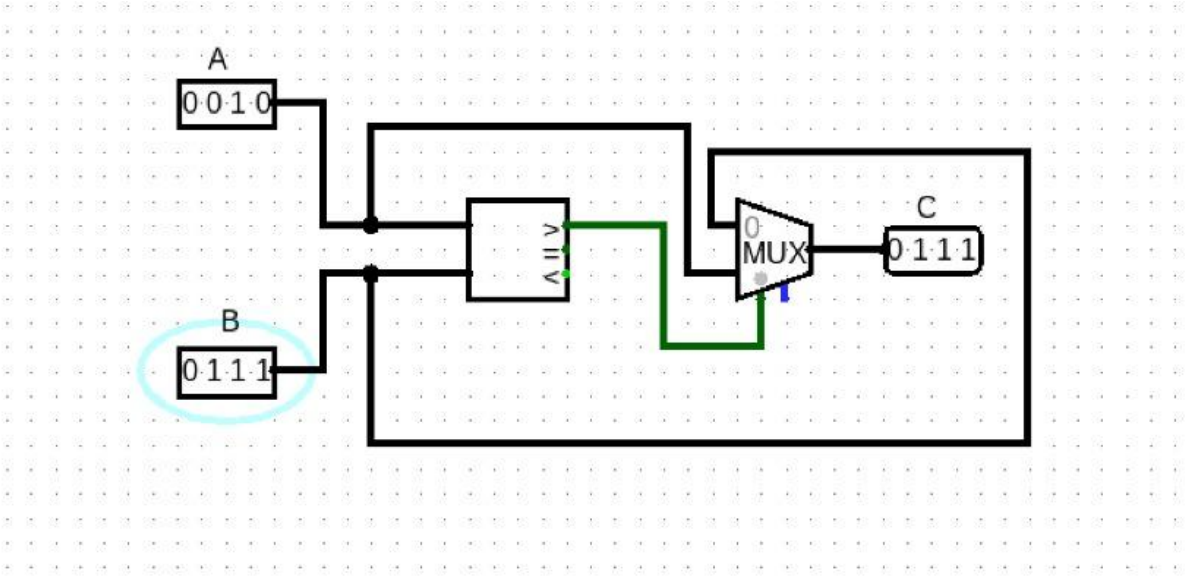
- **Objective:** Use an AI-assisted approach to translate a code snippet or conditional expression into its equivalent gate-level and block-level hardware realization, simulate it in Logisim, verify its truth table across all input combinations, and document the design.
- **System Components:**
  - **Digital Comparator:** Evaluates relational operators ($A > B$, $A = B$, $A < B$), outputting a selection/control signal.
  - **Multiplexer (MUX):** Acts as hardware implementation for the conditional branch selection (if-else), selecting between data inputs A and B depending on the comparator's output flag.

Part I: Circuit 1 Analysis (Conditional Maximum Selector — Scenario 1)

3.1. Objective & Circuit Overview

- Circuit 1 evaluates inputs A and B to implement a conditional selection equivalent to finding the maximum value or executing an if-else block.
- **Input Values (as per schematic):**
  - $A = 0010$ (Decimal 2)
  - $B = 0111$ (Decimal 7)

❖ **CIRCUIT IMPLEMENTATION IN LOGISIM:**



3.2. Output Verification & Test Cases

- **Comparison Phase:** The comparator checks if $A > B$ $(2 > 7)$, which evaluates to 0 (False).
- **MUX Routing:** Because the relational flag is 0, the multiplexer selects input corresponding to the alternative branch (B).
- **Observed Final Output (C):** $C = 0111$ (Decimal 7).
- *Verification:* The circuit successfully replicates the conditional logic where the larger value between A and B is routed to output C.

3.3. Discussion & Observations

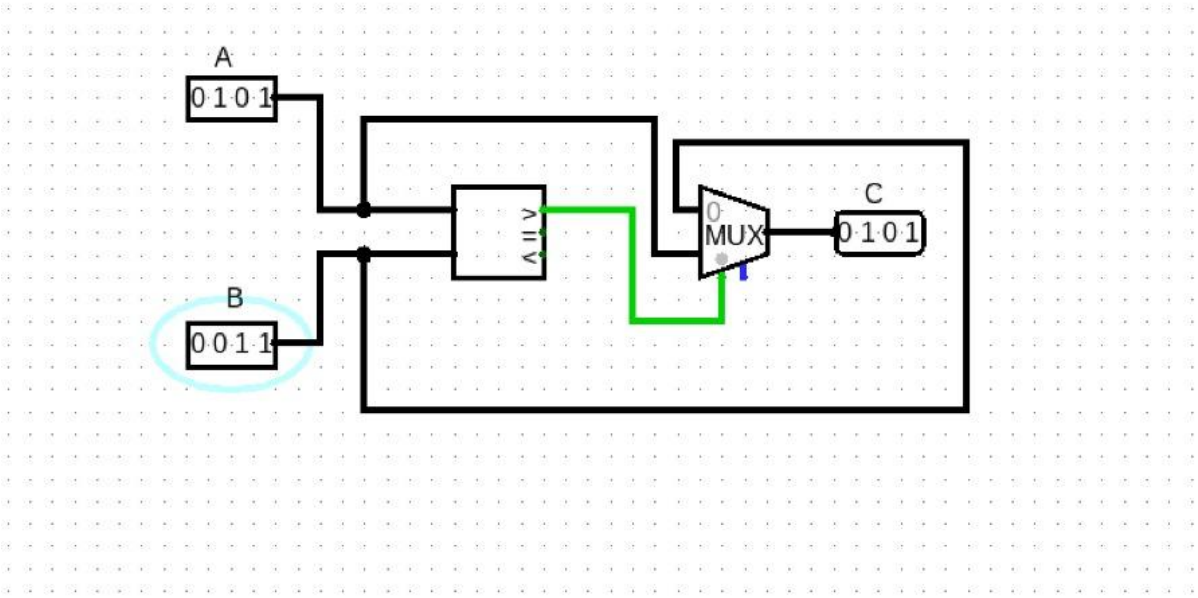
- **Relational Mapping:** The comparator efficiently eliminates the need for complex combinatorial gate arrays by providing a direct binary flag $(>, <, =)$.
- **Data Flow Control:** The MUX acts cleanly as a hardware switch, ensuring zero propagation delay conflicts during data selection.

Part II: Circuit 2 Analysis (Conditional Selection — Scenario 2)

4.1. Objective & Circuit Overview

- Circuit 2 tests an alternate set of input values to verify the robustness of the conditional branching hardware across different relational outcomes.
- **Input Values (as per schematic):**
  - $A = 0101$ (Decimal 5)
  - $B = 0011$ (Decimal 3)

❖ **CIRCUIT IMPLEMENTATION IN LOGISIM** :



4.2. Output Verification & Test Cases

- **Comparison Phase:** The comparator evaluates $A > B$ $(5 > 3)$, which yields a logical 1 (True).
- **MUX Routing:** With the selection line driven high (1), the multiplexer routes input A through to the output node.
- **Observed Final Output (C):** $C = 0101$ (Decimal 5).
- *Verification:* The output correctly matches input A, verifying proper operation when the conditional statement evaluates to true.

4.3. Discussion & Observations

- **Dynamic Adaptability:** Changing inputs instantly shifts the MUX routing path, demonstrating how high-level software branches map seamlessly to hardware multiplexing.
- **AI-Assisted Design Validation:** The generated architecture successfully matches theoretical expectations without redundant logic gates, confirming the accuracy of the automated design methodology.

5. Conclusion

Both circuits successfully demonstrate the hardware translation of high-level conditional code constructs using integrated comparators and multiplexers. Output verification confirms accurate data routing across various input states, fulfilling all functional requirements for AI-assisted digital system design.